

DEPARTMENT OF COMPUTER SCIENCE, UET LAHORE



OOP-THEORY-PROJECT

Title:

**UNIVERSITY TRANSPORT MANAGEMENT
AND BILLING SYSTEM**

Submitted By:

Ammar Bin Yasir (2025-CS-560)
Muhammad Asad Mukhtar (2025-CS-583)

Submitted To:

Ma'am Sana

Date Of Submission:

08/06/2025

SECTION 1: EXECUTIVE SUMMARY & SYSTEM SCOPE.....	2
1.1 Project Overview	3
1.2 Problem Statement.....	3
1.3 System Boundary Specifications.....	3
SECTION 2: OBJECT-ORIENTED ARCHITECTURE & DESIGN PATTERNS	4
2.1 The Polymorphic Inheritance Hierarchies	4
2.1.1 The User Authentication Dimension	4
2.1.2 The Fleet Logistics Dimension	4
2.2 Structural Module Relationship Models	4
2.2.1 Composition (The Financial Link)	5
2.2.2 Aggregation (The Central Coordinator).....	5
2.3 System Architecture Class Diagram	5
2.3.1 User Authentication & Accounts	6
2.3.2 Fleet & Route Management	6
2.3.3 Pass Applications, Billing & Reports	8
2.3.4 System Controller Engine & Storage	8
SECTION 3: FILE PERSISTENCE & SERIALIZATION STRATEGY	10
3.1 Chronological Loading Dependency Timeline	10
3.2 Polymorphic Type-Tagging Pattern	10
3.3 Active Save Execution Model	10
SECTION 4: MANUAL MEMORY MANAGEMENT MATRIX	11
4.1 Dynamic Array Expansion Rules (growList Algorithm)	11
4.2 The Net-Zero Leak Sweep Destructor	11
Section 5: Console UI/UX Specifications	11
5.1 Structured Box-Drawing Grid Parameters	11
5.2 Dynamic Spreadsheet Tabulation Matrix	12
5.3 High-Reliability Input Stream Defenses	12

SECTION 1: EXECUTIVE SUMMARY & SYSTEM SCOPE

1.1 Project Overview

The University Transport Management System is an enterprise-grade console application designed to handle the logistical, financial, and administrative operations of a multi-vendor transit network. Tailored specifically for complex institutional layouts like UET Lahore, this system bridges the operational gap between separate independent transport vendors and the university administration. The platform tracks dynamic assets, indexes transit infrastructure, manages student account records, and processes financial billing datasets within a unified command-line environment.

1.2 Problem Statement

Traditional campus transport networks suffer from massive inefficiencies due to fragmented data management. Independent shuttle and van vendors operate without a centralized data registry, leading to several critical failures:

- **Capacity Overruns:** Administrators cannot verify passenger saturation in real time, causing safety issues when vehicles exceed structural occupancy bounds.
- **Data Vulnerability:** System crashes or sudden terminal terminations result in total data loss for active financial registers and route mappings.
- **Analytical Blindness:** Manual tracking prevents administrative teams from generating dynamic revenue profiles or recognizing under-utilized routes.

This system addresses these challenges by acting as a centralized "**Single Source of Truth.**" It coordinates validation checks across all tracking tables, links independent components in memory, and provides structural durability through an automated file persistence database core.

1.3 System Boundary Specifications

To maintain high performance and architectural clarity, the system scope is explicitly defined:

- **In-Scope:** Centralized memory aggregation using raw pointer maps; polymorphic runtime execution dashboards; sequential dependency data streaming; instant active save serialization; defensive user input validation.
- **Out-of-Scope:** Real-time hardware GPS map tracking; external online banking integration; multi-threaded network sockets.

SECTION 2: OBJECT-ORIENTED ARCHITECTURE & DESIGN PATTERNS

2.1 The Polymorphic Inheritance Hierarchies

To achieve absolute decoupling and scalability, the application leverages compile-time structural contracts and runtime polymorphism across two major functional dimensions.

2.1.1 The User Authentication Dimension

The account management system is built upon an abstract base class `User`, which dictates the basic properties of identity (username, password). This base class establishes a pure virtual execution contract: `virtual void showMenu(TransportManager& manager) = 0;`.

During the operational loop, the central engine handles logged-in accounts using a single base pointer (`User* loggedInUser`). At runtime, C++ identifies the exact subclass type via late-binding. If an administrator authenticates, the system invokes `Admin::showMenu()`, rendering the ten system management modules. If a student authenticates, it routes execution to `Student::showMenu()`, rendering the client transportation dashboard. This eliminates nested condition checks and keeps the main application loop completely modular.

2.1.2 The Fleet Logistics Dimension

Fleet management is governed by the abstract base class `Vehicle`, capturing parameters like `vehicleID`, `totalCapacity`, `activePassengers`, and `assignedRouteID`. It enforces a virtual contract through virtual double `calculateMaintenance() const = 0;`.

The concrete subclasses, `Bus` and `Van`, inherit these base attributes but override the maintenance calculations to reflect their real-world economic profiles. For example, a `Bus` calculates overhead based on flat heavy diesel operations and large seating wear-and-tear, while a `Van` evaluates costs based on quick commuter metrics. This allows the financial reporting engine to process the entire fleet polymorphically within a simple loop using a `Vehicle` base pointer array.

2.2 Structural Module Relationship Models

2.2.1 Composition (*The Financial Link*)

The relationship between a student's transportation pass and their financial billing invoice is governed by strict **Composition**. Inside the private attributes of TransportPass, a physical instance of the Bill object is embedded directly without the use of pointers (Bill invoice;).

This ensures a tight lifecycle coupling between the two entities:

1. A Bill cannot exist independently in memory without a valid, tracking TransportPass.
2. When a student cancels their registration and the manager deletes the TransportPass object from the heap, C++ automatically triggers the immediate destruction of the embedded Bill instance, eliminating any risk of orphaned memory leaks.

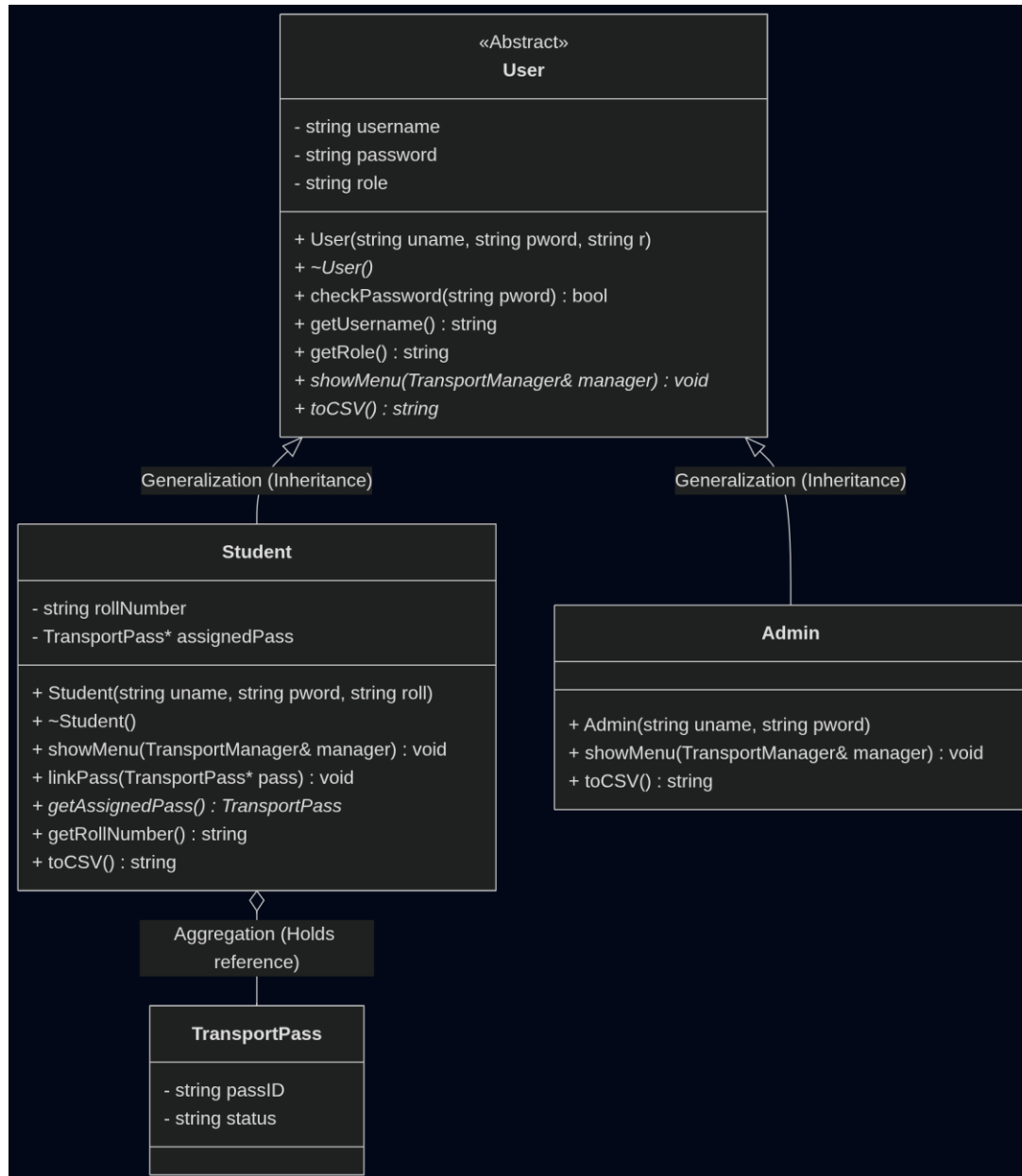
2.2.2 Aggregation (*The Central Coordinator*)

The system core is organized around the **Facade Controller** design pattern, implemented via the TransportManager class. The manager maintains master arrays of pointers targeting the active heap entities (User userList, Vehicle vehicleList, etc.).

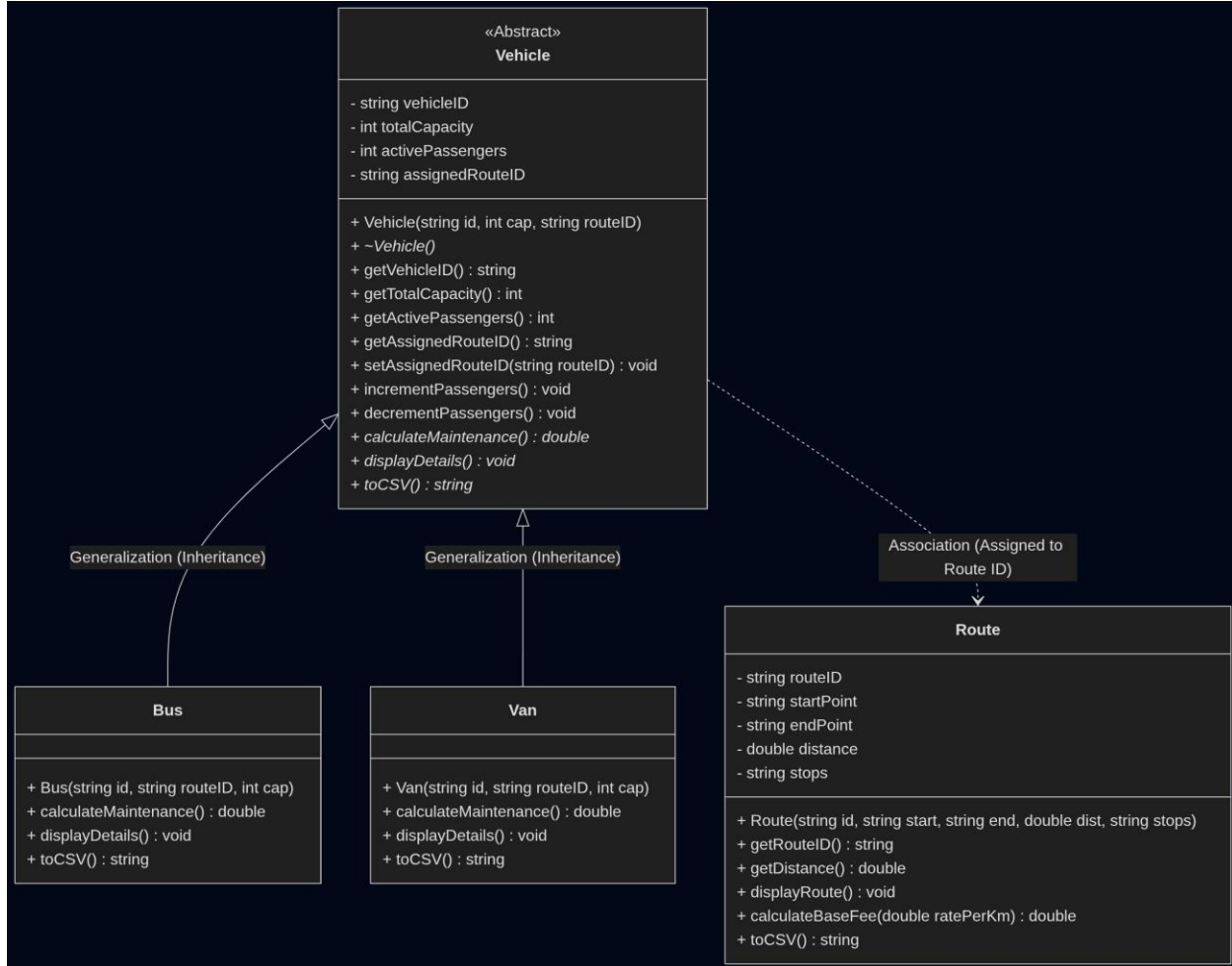
Because the manager owns these pointer tracking tables but does not tightly own the internal life of the individual objects, this represents an **Aggregation** model. The manager serves as the central brain, handling all cross-module business rules. For instance, when processing an application, it reads data fields from a TransportPass, cross-references the matching Route inside the route registry, verifies the structural seat limit on the corresponding Vehicle, and updates the student's internal links safely.

2.3 System Architecture Class Diagram

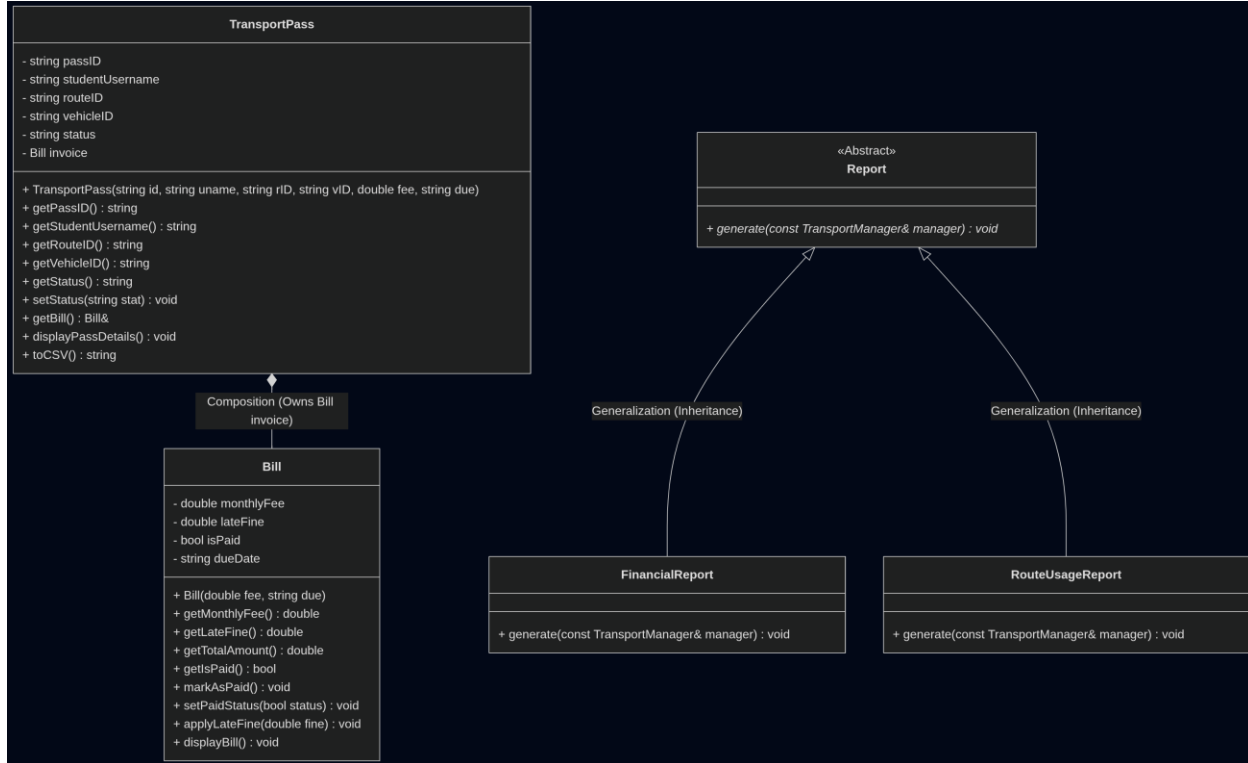
2.3.1 User Authentication & Accounts



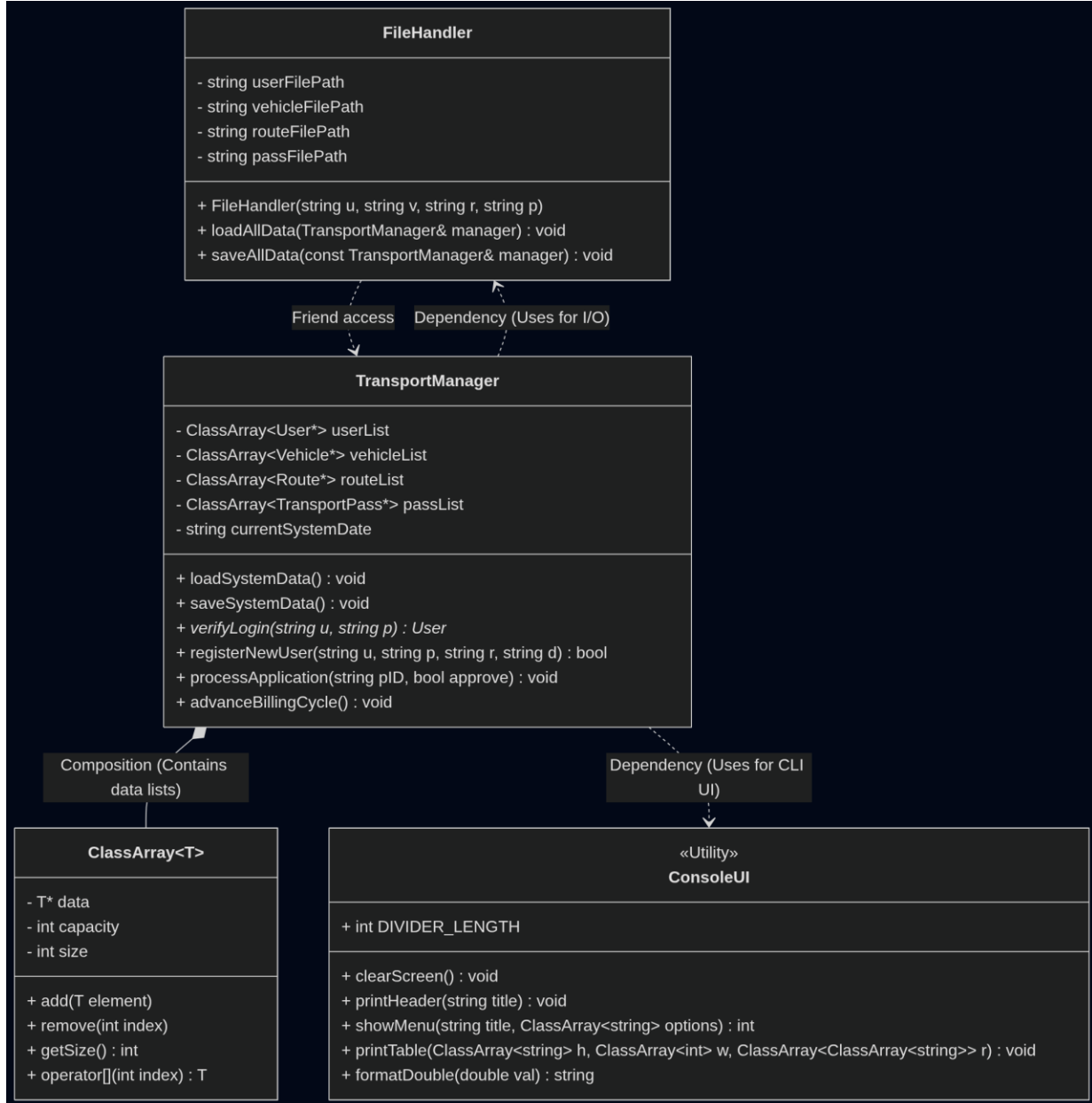
2.3.2 Fleet & Route Management



2.3.3 Pass Applications, Billing & Reports



2.3.4 System Controller Engine & Storage



SECTION 3: FILE PERSISTENCE & SERIALIZATION STRATEGY

3.1 Chronological Loading Dependency Timeline

Because the entity modules are closely interdependent, the FileHandler component cannot read database text files arbitrarily. It enforces a strict, sequential loading timeline during the system boot sequence:

[BOOT] → loadRoutes() → loadVehicles() → loadUsers() → loadPasses() → [RAM READY]

- **routes.txt Loads First:** Establishes the core geographical transit paths.
- **vehicles.txt Loads Second:** Validates and assigns vehicles to the pre-loaded Route IDs.
- **users.txt Loads Third:** Populates the administrative and student credential tables.
- **passes.txt Loads Last:** Reconstructs registration requests. This step requires that the corresponding Student, Vehicle, and Route pointers are already active in RAM so the manager can securely link the memory addresses together.

3.2 Polymorphic Type-Tagging Pattern

To bypass the compile-time hazard of **Object Slicing** during file reading operations, the system implements a Type-Tagging pattern. Flat text database rows utilize a unique comma-separated value (CSV) identifier prefix:

When FileHandler::loadAllData() parses a line, it extracts this first string token. The application evaluates this tag within an explicit condition switch *before* allocating heap space. If the token reads STUDENT, it routes parsing variables directly to new Student(). This ensures that derived-class parameters (such as a student's unique rollNumber) are fully preserved in dynamic memory, preventing object slicing.

3.3 Active Save Execution Model

To maximize data durability, the system avoids relying on a single data dump when the application closes. Instead, it utilizes an **Active Save Model**. Whenever an administrative or student operation successfully mutates system states (such as registering a new account, editing transit distances, or approving an invoice), the engine instantly fires FileHandler::saveAllData().

The function clears the old text records using the std::ios::trunc file mode and requests every live object to map its attributes back to disk via their virtual toCSV() methods. This

ensures that even if a terminal window is unexpectedly closed, data integrity remains intact.

SECTION 4: MANUAL MEMORY MANAGEMENT MATRIX

4.1 Dynamic Array Expansion Rules (growList Algorithm)

To meet advanced C++ engineering criteria without relying on the automated wrappers of `std::vector`, the `TransportManager` manually handles dynamic resizing. When an element counter peaks at its capacity limit (e.g., `userCount == userCapacity`), the manager executes a private list expansion utility:

1. Allocate new pointer array on the heap: [New Capacity = Old Capacity * 2]
2. Run loop copying memory addresses from Old Array to New Array
3. Execute 'delete[] oldArrayContainer' to free pointer slot memory
4. Point master tracker array to the newly expanded heap memory address block

This custom growth model prevents buffer overflow crashes while maintaining quick pointer access across all entity arrays.

4.2 The Net-Zero Leak Sweep Destructor

To guarantee zero memory leaks upon application termination, the `TransportManager` acts as a custom garbage collector within its destructor (`~TransportManager()`). It executes structured loops that iterate through every pointer slot up to the active item counters, explicitly invoking `delete`.

Because the base classes use virtual destructors (`virtual ~User() = default;`), C++ safely climbs down the inheritance lines during this sweep. It cleans up the derived allocations of `Student` and `Admin` completely, returning a pristine memory score to the operating system.

Section 5: Console UI/UX Specifications

5.1 Structured Box-Drawing Grid Parameters

The presentation layer implements a deterministic, bounded text grid based on a strict `DIVIDER_LENGTH` constraint of **100 characters**. The system structures every console view within border:

- **Header Bounding Enclosures:** Formatted utilizing `┌` top tracks, explicit tracking computations to calculate exact horizontal text layout balances, and matching trailing padded sequences.

- **Component Encapsulation Fields:** Menus and informational fields are wrapped dynamically inside left-to-right box channels (|), ensuring that no matter how long an input option string is, column boundaries never deform.

5.2 Dynamic Spreadsheet Tabulation Matrix

The `ConsoleUI::printTable` method bypasses primitive layout schemes by dynamically compiling structured data matrix layouts. The system maps distinct grid cross-sections leveraging variable spacing metadata arrays (widths and headers passed as explicit template arrays):

- **Header Intersects (└, ┌):** Explicitly bridges split data tracking columns at the absolute character intercept.
- **Data Row Intersects (├, │, ┘):** Enforces a clean spreadsheet layout inside the console viewport, allowing fluid row padding using left-aligned `std::setw()` stream targets.

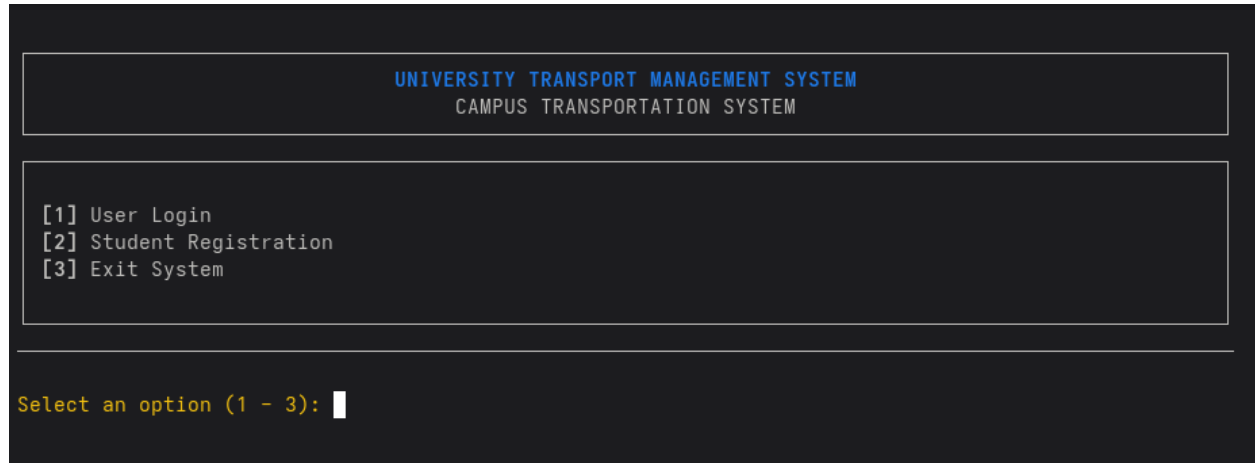
5.3 High-Reliability Input Stream Defenses

The input utilities implement high-reliability defensive architectures to isolate the backend from runtime stream failures:

- **Carriage Return (\r) Mitigation:** Inside `promptString()`, an explicit filter strip runs `input.pop_back()` if a trailing carriage return is detected, protecting the parser from file-redirection or platform-specific encoding variations.
- **End-of-File (ss.eof()) Stream Isolation:** Inside `promptInt()` and `promptDouble()`, raw input states are isolated inside localized `std::stringstream` workspaces. The validation sequence `ss >> value && ss.eof()` checks that the user entry begins with a valid numeric structure *and* that no malicious or trailing text exists to contaminate memory registers.

Section 6: Verification & Execution Testing

Phase 1: Boot & Authentication Phase



Phase 2: Administrative Control Actions

UNIVERSITY TRANSPORT MANAGEMENT SYSTEM
ADMINISTRATOR CONTROL PANEL

- [1] Add Route
- [2] Add Vehicle
- [3] Update Vehicle Details (Capacity/Route)
- [4] View All Vehicles
- [5] View All Applications
- [6] Process Pass Applications
- [7] Generate Route Usage Report
- [8] Generate Revenue Report
- [9] Advance Billing Cycle (New Month)
- [10] Set System Date
- [11] Logout

Select an option (1 - 11):

Phase 3: Student Interface Operations

UNIVERSITY TRANSPORT MANAGEMENT SYSTEM
STUDENT DASHBOARD - AMMARBINYASIR

- [1] View Available Routes
- [2] Apply for Transport Pass
- [3] View My Active Pass
- [4] Pay Pass Fee
- [5] Cancel Transport Registration
- [6] Logout

Select an option (1 - 6):

i